On the Usefulness of Automatically Generated Microservice Architectures

Luiz Carvalho, Thelma Elita Colanzi, Wesley K. G. Assunção, Alessandro Garcia, Juliana Alves Pereira, Marcos Kalinowski, *Member, IEEE*, Rafael Maiani de Mello, Maria Julia de Lima, and Carlos Lucena

***Abstract*—The modernization of monolithic legacy systems with microservices has been a trend in recent years. As part of this modernization, identifying microservice candidates starting from legacy code is challenging, as maintainers may consider many criteria simultaneously. Multi-objective search-based approaches represent a promising state-of-the-art solution to support this decision-making process. However, the rationale to adopt each microservice candidate automatically identified by these approaches is poorly investigated in industrial cases. Furthermore, studies with these approaches have not carefully investigated how maintainers reason and make decisions when designing microservice architectures from legacy systems. To address this gap, we conducted an on-site case study with maintainers of an industrial legacy system to investigate the usefulness of automatically generated microservice architectures. We analyze design decisions pointed out by the maintainers when reasoning about microservice candidates using several criteria at the same time. Our study is the first to assess a search-based approach involving actual maintainers conceiving microservice architectures in an industrial setting. Therefore, firstly, we considered individual evaluation of microservice candidates to understand the rationale for identifying a service. Secondly, we conducted a focus group study with maintainers with the goal of investigating design decisions at an architectural level. The results show that: *(i)* the automated approach is able to identify useful microservices; *(ii)* the criteria observed by previous studies are, in fact, considered by maintainers; and *(iii)* the maintainer profiles, i.e., the preferred granularity for microservice, highly affect design decisions. Finally, we observed the maintainers needed little effort in adjusting the automatically identified microservices to make them adoptable. In addition to indicating a promising potential of search-based approaches to generate microservice architectures, our findings highlight the need for: *(i)* interactive and/or customizable approaches that enable maintainers to include their preferences during the search process, and *(ii)* flexible or automated selection of criteria that fits the scenario in which the modernization is taking place.**

***Index Terms*—Microservice architecture, search-based software engineering, microservice identification, legacy system, qualitative study.**

Manuscript received 26 September 2022; revised 4 December 2023; accepted 22 January 2024. Date of publication 1 February 2024; date of current version 15 March 2024. This work was supported in part by CNPq under Grant 151723/2020-6, Grant 428994/2018-0, Grant 434969/2018-4, Grant 312149/2016-6, Grant 309844/2018-5, Grant 421306/2018-1, Grant 312827/2020-2, Grant 152179/2020-8, and Grant 408356/2018-9; in part by CAPES under Grant 175956 and Grant 473590/2020-00; in part by FAPERJ under Grant 22520-7/2016, Grant 010002285/2019; in part by PDR-10 Fellowship under Grant 202073/2020; and in part by FAPPR under Grant 51152 and Grant 51435. Recommended for acceptance by M. Mirakhorli. *(Corresponding author: Wesley K. G. Assunção.)*

Luiz Carvalho, Alessandro Garcia, Juliana Alves Pereira, Marcos Kalinowski, and Carlos Lucena are with the Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro 38097, Brazil (e-mail: lmcarvalho@inf.puc-rio.br; afgarcia@inf.puc-rio.br; jpereira@inf.puc-rio.br; kalinowski@inf.puc-rio.br; lucena@inf.puc-rio.br).

Thelma Elita Colanzi is with the Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro 38097, Brazil and also with the State University of Maringá, Maringá 87020-900, Brazil (e-mail: thelma@din.uem.br).

Wesley K. G. Assunção is with the Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro 38097, Brazil and also with North Carolina State University, Raleigh 27607 USA (e-mail: wguezas@ncsu.edu).

Rafael Maiani de Mello is with the Federal University of Rio de Janeiro, Rio de Janeiro 21941-594, Brazil (e-mail: rafaelmello@ic.ufrj.br).

Maria Julia de Lima is with the Tecgraf Institute, Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro 38097, Brazil (e-mail: mjulia@tecgraf.puc-rio.br).

Digital Object Identifier 10.1109/TSE.2024.3361209

I. Introduction

LEGACY systems are commonly found in industry and represent a long-term and massive investment [1], [2]. Despite their business importance, these systems often depend on obsolete technologies and usually have a monolithic architecture, which harms the introduction of innovation [3], [4]. Moreover, the maintenance of legacy systems is inherently expensive as features commonly suffer from poor modularization, tangled with each other within various modules [1], [5].

To deal with such issues, several companies have been adopting microservices to modernize legacy systems [3], [4], [6], [7], [8], [9], [10]. In this modernization, there are various challenges. Separating the code of a legacy system into small units to create the microservices is reported as one of the main challenges faced by developers [4], [5], [11]. Yet, for identifying adequate microservices, developers must examine a legacy code and make many complex decisions considering several criteria at the same time [12]. For example, a developer may desire that extracted microservices have a low network overhead. The same developer may consider the cohesion of each microservice as an important criterion. Thus, such a developer intends to maximize the cohesion of the extracted microservices and keep the network overhead at acceptable levels. Previous studies observed that experienced developers usually consider at least four criteria as simultaneously useful for identifying microservices from legacy systems [12], [13], [14].

Despite the practical need for microservice identification from legacy systems, there is a limited number of industrial

0098-5589 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

investigations focusing on this activity [3], [4], [15]. Moreover, the majority of studies are only based on the manual analysis of the source code and other artifacts of the legacy system from the point of view of useful criteria. Further, these studies do not explore the reasoning process of maintainers during the analysis to define a microservice architecture. By maintainers, we refer to developers improving and evolving the source code by means of the re-design of certain legacy features into microservices.

As the manual identification of microservices is challenging [12], [16], a strategy to support the automated microservice identification is using search-based approaches [17], [18], [19], [20], [21], [22], [23], [24], reducing the maintainers' effort and allowing the consideration of multiple criteria simultaneously. Although search-based approaches present promising results, there is no on-site case study (*i.e.*, a study considering a real-world/industrial legacy system together with its context and maintainers) investigating the usefulness of architectures generated by search-based approaches. Likewise, there is a lack of investigation into the rationale and tasks performed by the maintainers for the decision-making process. Even worse, search-based approaches are usually assessed in the context of: *(i)* simple, academic, already well-modularized systems [25], [26]; or *(ii)* projects, usually open-source, where the decision-making process of maintainers is not observed [17], [18], [19], [20], [22], [24], [27], [28].

Another fundamental part of the decision-making process is to define the size of microservices. By definition, microservices are highly decoupled and cohesive pieces of software, with a clear responsibility, that work together using lightweight network protocols [11]. However, there is a shortage of studies investigating what "micro" in microservices means when defining microservice granularity [11], [29]. In our previous work [21], we have observed a divergence in the maintainers' opinion regarding the level of granularity for a microservice. Such a divergence is justified because maintainers may have different profiles. These profiles refer to personal preferences regarding microservice granularity. On the one hand, some maintainers clearly prefer coarse-grained "micro"services, and on the other hand, some maintainers prefer fine-grained "micro"services. Since the definition of the granularity of microservices is unclear, it is essential to investigate how automated approaches behave in practical scenarios. In addition to pointing to potential limitations of existing search-based approaches, such an investigation can also bring insights for improving existing automated support. However, to the best of our knowledge, there is no work in the literature performing an in-depth investigation on how the profile of maintainers influences the decision-making process when reasoning about microservice architectures.

Motivated by the limitations presented above, we conducted an on-site case study to address gaps in existing studies and provide empirical evidence on the usefulness of automated approaches. In addition, we observe the discussions and rationale of maintainers when analyzing automatically generated microservices. For that, our study relies on an industrial legacy system and its maintainers. The aim is threefold: *(i)* investigate the potential adoption of automatically generated microservice architectures from the maintainers' point of view, *(ii)* observe how maintainers make design decisions on these architectures to obtain an adoptable architecture, and *(iii)* extract insights to enhance automated approaches. Our study is the first one carried out in the industry to assess the usefulness of microservice architectures generated by search-based approaches.

Our study encompasses two main steps. In the first step, we inquired maintainers about microservice candidates identified by a search-based approach [21]. The main goal of this first step is to characterize how maintainers evaluate the potential adoption of the automatically generated solutions. To do so, we observed the practical usage of several criteria in order to evaluate whether they are used to assess the adoptability of each microservice candidate. In this work, adoptability refers to which degree a microservice candidate is considered useful for the practice [21]. We also observed the influence of different maintainers' profiles on their evaluations. We used the results of this step to improve the search-based approach adopted in the study before performing the next step.

In the second step, we conducted a focus group [30] to evaluate the automatically generated microservice architectures, which includes the maintainer's feedback during the search process. The maintainers were also asked to change the architectures to fit the modernization needs. We observed the groups of maintainers gathering and discussing multiple perceptions before they reached a final decision. Thus, we collected maintainers' feedback, identified potential problems, and learned some lessons.

The results of our case study enrich the state of the art as they allow enhancing existing search-based approaches for similar purpose from findings gathered from industry. In summary, the key findings of our study are:

1) *The adoption of solutions is highly dependent on the maintainers' preferences.* Surprisingly, our results reveal that the maintainers of the case study have diverging preferences about the granularity of microservices. We identified different profiles, namely maintainers who prefer fine-grained microservices and maintainers who consider coarse-grained microservices as the best ones. Hence, an adaptive search-based approach, which includes interaction as part of the optimization process to capture the maintainer profile, is able to generate solutions that are more useful in practice.
2) *Automatically generated microservice architectures are a good starting point.* The solutions generated by the search-based approach were modified by maintainers to become practically useful. Only a few modifications were required to adopt the solutions in practice. Interestingly, despite being a complex and highly expert-dependent task, we observed that the automatically generated solutions are a good starting point to support microservice identification.
3) *Existing automated approaches must be improved to deal with industrial needs.* Companies facing the modernization of legacy systems using microservice architectures may have different needs and goals. This leads to different modernization scenarios. Thus, to allow these companies to take advantage of automatically generated

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

solutions, search-based approaches must offer flexible or automated selection of criteria that fits the scenario in which the modernization is taking place.

The remainder of this paper is organized as follows. Section II describes studies with automated approaches to identify microservices based on legacy code and discusses their limitations. Section III presents the design of the qualitative studies reported in this paper. Section IV presents the evaluation of the individual interviews. Section V presents and discusses the results of the focus group. Sections VI and VII present the lessons learned and the threats to validity, respectively. Section VIII discusses the related works, and Section IX concludes the paper.

II. Background

Microservices are small and autonomous services that work together by using lightweight communication protocols [11]. The notion of small refers to the need for producing services with single responsibilities [29]. The notion of autonomous services implies that each microservice should be highly decoupled [11], [29]. In the same vein, lightweight protocols are adopted to improve decoupling further. In fact, a well-designed microservice architecture promotes benefits, even if this architecture is migrated from a legacy system. Among the benefits, developers highlight the reduced effort for maintenance and evolution, increased availability of services, ease of innovation, continuous delivery, ease of DevOps incorporation, and facilitated scalability [4], [5], [31].

Microservice identification is a process usually performed as manual work by maintainers [4], [5], [15], [32], [33]. This process is laborious, error-prone, and expensive. Then, automated approaches have been proposed to reduce these drawbacks. The automated approaches analyze legacy systems with the goal of automatically identifying microservices [17], [18], [19], [20], [21], [25], [27], [28]. They also determine which legacy code elements may be used to support the implementation of each microservice.

There are certain criteria to support decisions on deriving a microservice architecture from a legacy system. In this work, a criterion is an aspect used to support the decision-making process to migrate (either partially or fully) the legacy system in a microservice architecture. Each criterion requires the extraction of information from either the structure or execution of the legacy system. Most previous automated approaches to identify microservice candidates in legacy code are based on a few key criteria. They usually adopt coupling and cohesion, two classic modularization criteria [17], [18], [20], [25], [27], [28]. Only recently, we introduced *toMicroservices*, an automated approach that adopts five criteria in a simultaneous way [21]. Thus, providing a more realistic approach, since the adopted criteria have their importance recognized by developers [12].

The identification of microservices in legacy systems is a multi-objective optimization problem, as several criteria must be taken into account. To tackle this task, Search-Based Software Engineering (SBSE) [34], [35] is commonly adopted [18], [20], [21]. In SBSE, optimization algorithms are used to address software engineering problems [36], dealing with the difficult task of balancing competing (and sometimes inconsistent) constraints. SBSE techniques search for acceptable solutions for situations where perfect solutions are either theoretically impossible or practically infeasible [34].

In this study, we used *toMicroservices*, an SBSE approach [21]. This approach uses a many-objective optimization algorithm to simultaneously optimize five criteria for identifying microservice candidates. These criteria are coupling, cohesion, feature modularization, network overhead, and reuse. The approach requires three pieces of information as input: *(i)* the legacy source code, *(ii)* a list of features, and *(iii)* the number of microservices to be identified. By feature, we refer to a distinguishing characteristic of a software product. The concept of "feature" allows a consistent abstraction to be employed when making decisions for microservice candidates. The approach analyzes the legacy's method structure, to enable the identification of method-level crosscutting features as microservice candidates. *toMicroservices* uses a graph-based representation, where each vertex represents a method of the legacy system assigned to its respective feature. Each edge contains static and dynamic information. The output artifacts generated by *toMicroservices* are: *(i)* a set of candidate solutions, where each element is a graph of microservice candidates, and *(ii)* the source code in the legacy system associated with each candidate. Each generated solution is a candidate microservice architecture for the legacy system. For an example of the *toMicroservices* graph-based representation, see [21], [37].

III. Study Design

The usefulness of search-based approaches to identify microservice candidates is poorly investigated in industrial scenarios. Thus, we conducted an empirical study with a twofold goal: *(i)* evaluate the adoptability of microservice architectures generated by an automated approach, namely *toMicroservices* [21], and *(ii)* investigate the design decisions made by maintainers to obtain an adoptable architecture.

Our empirical case study has two steps. In the first step, we conducted an individual interview with maintainers to understand how they evaluate the adoptability of single microservice candidates (see Section IV). We investigate the criteria they used, their profile regarding granularity level, features recognized, and how they consider microservice candidates adoptable based on the solutions generated by the automated approach. In a second step, we conducted a focus group with maintainers to evaluate the adaptation of microservice architectures, focusing on the decision-making process to make an architecture adoptable (see Section V). Focus group is a qualitative research method based on gathering data through the conduction of group interviews, called sessions [30]. Focus group studies have been conducted in the field for assessing new technologies, identifying lessons learned from previous studies [38], exploring emerging research topics [39], gathering maintainers' feedback about research questions [30], and characterizing the maintainers' decision-making process [40].

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

A. Goal and Research Questions

The goal of this work is *to investigate the usefulness of microservice architectures generated by a search-based approach for practical adoption in light of the maintainers' perspective*. We aim to analyze maintainers' perspectives during the decision-making process about the adoption (or not) of generated solutions. For that, we devised the research questions (RQs) described below. *RQ1* and its sub-research questions are addressed by the first study involving individual interviews with maintainers to evaluate single microservices (see Section IV). Observing maintainers individually allowed us to understand in a more isolated way the adequacy, limitations, and how they evaluate the adoptability of a single microservice. Then, *RQ2* and its sub-research questions aim at going forward and analyzing the multiple perceptions of a group of maintainers to reach a consensus in the final architectural decision (see Section V).

RQ1 - How do maintainers assess the adoptability of microservice candidates? Software development activities are typically influenced by human aspects at different levels. Despite the advanced resources available for automatizing some software engineering activities, individuals still have the final word, mainly due to the uniqueness of the context surrounding each software project [41]. In this way, by answering *RQ1*, we intend to understand how maintainers evaluate the potential adoption of microservice candidates identified by a search-based approach. More specifically, we analyze how the scope of a microservice candidate and the maintainer's profile impact the microservice adoptability. Towards this end, we observe which criteria maintainers consider during the evaluations. Then, we presented a set of microservice candidates to maintainers of a legacy system, who evaluates the adoptability of microservices in the industrial case study. To that end, we center our attention on the following sub-research questions:

RQ1.1 - *How does the scope of a microservice candidate impact its adoptability?* In the context of our work, the scope consists of the microservice responsibilities, *i.e.*, features considered for its extraction. Thus, the scope of a microservice is well-defined when maintainers easily identify its responsibility by the methods it has. In practice, as much as a microservice has its scope well-defined, modularizing features, the more adoptable it is. In this sub-RQ, we observed how *each participant* evaluated the adoptability of the *single microservices* generated by our search-based approach. The aim is to understand the preference of maintainers for isolated microservices and whether the microservices are in accordance with the maintainers' profile.

RQ1.2 - *Which are the criteria considered for assessing the adoptability of an automatically generated microservice?* Criteria are key elements in the decision-making process to identify microservice candidates and evaluate their adequacy (see Section II). Despite the studies that asked maintainers to point out more criteria than coupling and cohesion, there is a lack of practical observations on the criteria actually used to evaluate the identified microservices. In this context, we conducted a careful observation of maintainers working on the evaluation of microservice candidates.

RQ2 - How do maintainers make design decisions together on microservice architectures? Since architectural decisions are typically made in groups, to answer this research question, we invited maintainers to evaluate microservice architecture solutions generated by the search-based approach. Thus, we analyzed how they made decisions in a realistic scenario to reach an adoptable microservice architecture from that one obtained by the search-based approach. This RQ is addressed in a focus group study presented in Section V. The focus group is designed to investigate the usefulness of the search-based approach with a realistic scenario and adopt an interactive process. To that end, we center our attention on the following sub-research questions:

RQ2.1 - *To which degree is the automatically generated architecture adoptable?* Software architectures are designed or modified in several phases of the software development life cycle, such as design, implementation, and testing. Thus, architectural decisions are commonly distributed in groups of maintainers. Different architectural elements are used in the decision-making process. This sub-RQ intends to evaluate the adoptability of automatically generated *microservice architectures* from the perspective of *groups of maintainers*. The goal is to evaluate the solutions generated by a search-based approach in a real scenario to gather evidence about their applicability and limitations.

RQ2.2 - *Which are the adaptations proposed by maintainers to improve candidate architectures?* There is a lack of knowledge on how developers discuss the design and how they reason about microservice architectures in practice. Thus, this sub-RQ aims to understand which adaptations are proposed by maintainers to change an automatically generated microservice architecture to better meet maintainers' needs. Furthermore, this RQ allows analyzing how different criteria influence maintainers' decisions, leading to adaptations in the architecture candidate.

RQ2.3 - *To what extent does the profile of maintainers influence their architectural decisions?* In the RQ above, we focus on the influence of criteria when maintainers propose adaptations in the architecture candidate. However, our previous study points out that maintainers have different preferences/profiles in terms of granularity when evaluating individual microservices [21], which might also affects the design decisions on the architectural level. In this RQ, we investigate the impact of different profiles in the design decisions during the evaluation of the candidate architecture. More specifically, we evaluate how the decision-making process can be affected by fine and coarse-grained profiles, and the differences between the groups.

B. Initial Setup

We performed a case study involving an industrial real-world legacy system. We follow two steps to obtain the microservice candidates and the microservice architectures that were evaluated in our empirical study: *(i)* eliciting industrial requirements for microservice identification; and *(ii)* selecting a search-based

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

approach to accomplish the industrial requirements. These steps are described in the next subsections.

1) Eliciting Industrial Requirements: We performed an on-site case study, to investigate solutions for one system whose maintainers are interested in migrating to a microservice architecture. To better understand the industrial requirements during a migration process of a legacy system to a microservice architecture, we performed interviews with the system maintainers.

Our case study relies on a Java legacy system in the domain of the oil and gas industry that has been maintained for more than 15 years. The system was developed and is maintained at Tecgraf Institute.[1] The maintainers of this system reported that its maintenance is laborious, very complex, and time-consuming, even to add a simple new feature. For example, as the system does not use a database, the search for keywords in different files is very slow. Moreover, the system exhibits all the legacy problems mentioned in Section I.

Microservice Identification Context. Aiming at mitigating the difficulties of maintaining the legacy system, its maintainers started analyzing its migration to a microservice architecture. Initially, the maintainers tried to manually analyze the source code to identify possible features to be migrated into microservices. However, this manual analysis required a high effort from maintainers due to system complexity and size. To reduce effort, the maintainers used visual tools based on graphs representing the legacy system classes and their relations. The maintainers stated that the use of such tools was not helpful due to the poor and complex modularization of the system features. As an alternative strategy, the maintainers also considered state-of-the-art automated approaches. However, they struggled to use them for the reasons mentioned in Section II, especially the need for considering several criteria during the microservice identification.

"Microservification" of certain legacy features. Microservification promotes the re-design of legacy features into microservices. The maintainers prioritized the "microservification" of three important features at the first stage: *Algorithm*, *Authentication*, and *Project*, described below. These features have at least two subfeatures each (revealed in our results), 180 classes and 1,038 methods associated with their implementation in the legacy code.

- *Algorithm*: provides algorithms information by a REST API, including parameters, binary, documents, and connection points with other algorithms. In addition, this information can be stored using different resources.
- *Authentication*: authenticates and provides information to system users. It includes token creation and validation, login and password verification, password updating, and providing users' basic information.
- *Project*: responsible for the concept of providing a collaborative environment for the system's users. This collaborative environment includes shared files and metadata between different users or types of users.

[1]https://www.tecgraf.puc-rio.br/ The system was designed to be a framework that allows customization to different customers. However, after many years of maintenance, its architecture degraded. Currently, the system has poor modularization, *i.e.,* high tangling and scattering of features, and large classes.

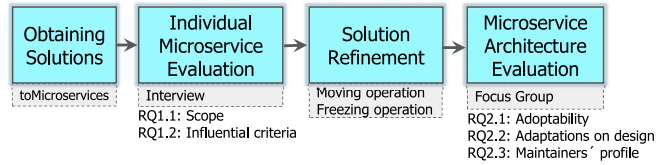


Fig. 1. Phases of the research method.

We consider that this industrial case study is similar to legacy systems that have been migrated to microservice architectures. Experience reports found in the literature [3], [4], [6], [8], [9] describe systems with similar characteristics: complex and time-consuming maintenance, monolithic architecture with components realizing tangled features, and need for customization and modernization.

2) Selecting the Search-Based Approach for the Study: As mentioned in Section II, most automated approaches consider only coupling and cohesion during the microservice identification. They also do not support the incremental migration to a microservice architecture. Hence, *toMicroservices* [21] is currently the only alternative approach that satisfies all the industrial requirements, namely: *(i)* the ability to simultaneously optimize many criteria, *(ii)* the fine-grained identification of microservices at method level, *(iii)* the support to the identification of microservices that improve feature modularization, and *(iv)* the possibility of identifying microservice candidates only for the features that were prioritized for an incremental migration process. In this sense, we consider that *toMicroservices* is the state-of-the-art search-based approach and we use it for microservice identification.

C. Execution Phases

Based on the initial setup and to achieve the goal of this work, we conducted the empirical study following the phases presented in Fig. 1 and described next.

1) Obtaining Solutions: In this section, we describe how the solutions were automatically generated using *toMicroservices*. We adopted the same parameter configurations used in our previous work [21], [37] to set up the algorithm NSGA-III. The population size was set to 100 individuals. The maximum number of iterations was set to 10,000, which is also the stopping criterion. There are also three parameters related to the problem: *(i)* the fraction of methods exchanged by the mutation operator, set to the minimum of 1 to the maximum of 50% of all microservice methods; *(ii)* the number of methods allocated in each microservice, set to between 3% and 16% of the total number of methods; and *(iii)* the number of microservice candidates, set to 5 and 10 based on the desired number of subfeatures the maintainers recognized in the "microservification". Thus, we designed two scenarios for our empirical study: **Scenario 1** – solutions considering the identification of 5 microservices; **Scenario 2** – solutions considering the identification of 10 microservices.

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

Given the randomness of genetic algorithms, we executed 30 independent runs for each scenario, according to guidelines for SBSE studies [42], [43]. Hence, after the 30 runs, we obtained two different sets of non-dominated solutions for each scenario. During the study execution, solutions from each scenario were selected according to the maintainer's profile. The procedures used to select the solutions are described in Sections III.C.2 and III.C.4.

2) Individual Microservice Evaluation: We performed a qualitative evaluation to understand how maintainers judge the adoptability or single microservice candidates from the point of view of criteria and their profiles. This section describes the qualitative study conducted to evaluate individual microservices, which involved interviewing some maintainers of the legacy system concerning their opinion about some single microservices generated by the search-based approach to answering *RQ1*. The following steps were conducted:

1) We invited eight maintainers involved in analyzing the microservification alternatives to a microservice architecture, to participate in the study (see Section IV).

2) We selected five solutions from the set of non-dominated ones found by the search-based approach. Each solution has the best value for one of the proposed criteria.

3) We asked the maintainers to indicate if they are specialists for each of the three features prioritized for "microservification", as follows: *Rate your knowledge about <feature>: Not knowledgeable (I do not know anything); Somewhat knowledgeable (I have a vague idea); Knowledgeable (I am familiar with it); Very knowledgeable (I know all/most classes and methods of it)*.

4) In the second part of the interview, the maintainers answered a questionnaire about how useful they found each criterion used by the search-based approach. In this questionnaire, we used the five-point Likert scale.

5) In the third part of the interview, maintainers answered about the adoptability of a set of microservices. During the interview, each maintainer assessed four microservices from two different solutions. The solutions were selected from the set of solutions created in Step 2, according to the maintainer expertise indicated in Step 3. The first selected solution has the best value of the criterion pointed as the most useful by the maintainer. The second one has the best value of the criterion pointed as the lowest useful. Given the selected solutions, two microservice candidates were chosen from each solution. For each microservice, we repeated the following process: we presented the microservice together with the code of the respective microservice and asked the maintainer: *Would you adopt this microservice? Why?* The adoptability was collected through a five-point Likert scale, and the motivation was summarized in a textual field. During the analysis of the solutions, we recorded the audio of the interviews and took notes about any comments made by the maintainers.

6) In the last step, the data about the criteria usefulness, microservice rates, and answers for the question of Step 5 were collected and analyzed by three of the authors.

3) Refining Solutions: Based on the knowledge acquired during the evaluation of single microservices (a detailed discussion of the results is presented in Section IV), we observed that maintainers often recommended similar refinements. When analyzing such refinements, we realized they could be part of the strategy to obtain better solutions during the evolutionary process of the search-based approach. We present below each of the refinement operations we defined.

- **Moving operation** changes a block/blocks of methods to another microservice, where this block should contain at least one method.
- **Freezing operation** prevents the selected methods from being moved by genetic operators during the next offspring generations after the operation is performed.

These operations allow the user interaction to adjust constraints to the obtained solutions before re-executing the automated approach to generate near-optimal microservice architectures. That is, the operations are applied by the user to a solution generated by the automated approach, and the solution is used to a new execution of the evolutionary process. In summary, the user can accept the solutions and decline the re-execution; otherwise, the approach requests that the user applies the refinement operations in the solution. The operations allow better exploring the maintainer's profile and knowledge about the system under analysis. The moving operation only changes the block of methods, while the freezing operation imposes constraints on the next offspring generations. The constraints imposed by the moving and the freezing operations can be modified in the next steps of refinement during new re-executions. For example, a developer can freeze an entire microservice or move some methods and freeze the additional methods insofar as they adopt the analyzed microservice. Thus, the next offsprings during the new re-execution will not change the block of methods frozen. The maintainers can change or maintain the operation in each new re-execution.

Re-executions of the approach using the refinement operations were performed to obtain solutions for the next phase of our study, as described in the next section. In our case study, the refinement operations were manually applied between independent runs. However, the refinement operations can be further used in automated approaches to put the user in the loop to refine solutions through generations during the optimization process.

4) Microservice Architecture Evaluation: In this phase of our study, we investigate the entire microservice architecture, presenting the solutions for groups of maintainers. This phase relies on the focus group method [30].

Composing the groups. Seven maintainers from the first study (interview) were able to participate in this study. They were divided into two groups, following the guidelines of focus group studies that recommend clustering the groups by similarities [30]. The composition of the groups was done by their profile answers collected in the previous study. Four maintainers (P1, P2, P6, and P8) presented a profile with a *fine-grained* microservice perspective since they positively evaluated microservices that modularized single subfeatures with few methods. The other three maintainers (P3, P5, and P7) have a *coarse-grained* microservice profile, where several subfeatures or a complete feature in a microservice

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EVALUATION OF THE MICROSERVICE CANDIDATES

Participant	Experience in the Legacy	Recognizable Features	New Features	Adoptability Rate for Microservices
Scenario 1: Architectures with 5 microservices				
P1*	6 months	5	2	3, 2, 4, 5
P2*	2 years	3	2	3, 2, 4, 1
P3**	2 years	4	1	2, 4, 2, 4
P4	20 years	7	6	1, 1, 1, 1
Scenario 2: Architectures with 10 microservices				
P5**	13 years	6	4	5, 3, 2, 1
P6*	8 years	4	2	1, 5, 1, 4
P7**	1 year	5	3	3, 3, 2, 4
P8*	3 years	5	3	2, 4, 4, 3

* *fine-grained* microservice profile, ** *coarse-grained* microservice profile

was indicated as adequate. Table I shows the experience of each maintainer.

The division by profile also helped to better configure the search-based approach to the particular needs or perspectives of each group (see Section III.C.1). In this context, the *coarse-grained* group received a solution where the automated approach was configured to generate five microservices (**Scenario 1**). The *fine-grained* group received a solution where the automated approach was configured to generate ten microservices (**Scenario 2**).

Artifacts. The artifacts presented to maintainers during the focus group session were: *(i)* visual representation of the microservice architecture, *(ii)* the relationships between methods of different microservices, where each method call is presented as a relationship, and *(iii)* the traceability to the legacy code at the level of methods. The visual representation contained the microservices and their relationships. The relationship is directional, and the weight in each relationship is the coupling between the microservices, in terms of the number of static calls between them.

Fig. 2 depicts the microservice architectures evaluated in the focus group. For the sake of legibility, relationships involving the microservices of user, log, and authentication were omitted. The future microservice architecture in the industrial case will contain an API Gateway that will request user and authentication microservices. The request header will have additional information related to basic user information, such as ID, name, role, and authentication status. Thus, the maintainers expect that every microservice presented in the figure already has access to this basic information.

Instrumentation and Execution. A moderator conducted a single focus group session with each group of maintainers. Each focus group section took one hour, including 10 minutes of introduction and 50 minutes of discussion. First, the moderator introduced the visual representation of the generated microservice architecture to the group. Second, the moderator explained the relationships between microservices, presented in the visual representation. The relationships between microservice candidates include the methods that call or are called from other microservice candidates. That is, they determine the provided and required interfaces of each microservice. Third, each microservice was described regarding its main responsibilities and subfeatures. Finally, the moderator stated that maintainers could request, at any moment, details of the relationships between microservices, the list of methods of each microservice, or the source code of each method.

After the initial explanation, the moderator asked maintainers to evaluate the architecture presented. During the discussions, the moderator requested each decision to be justified. The justifications included the operations required to modify the architecture, information based on their experience with the legacy code, or reasons for accepting or rejecting microservices and/or their relationships. The moderator took notes of the decision reported by the maintainers. Moreover, the entire meeting was recorded to confirm the notes and allow for more qualitative analysis.

At the end of the focus group session, the maintainers were asked about the adoptability of the microservice architecture generated by the search-based approach. They were also inquired about the adoptability of the final architecture—the one modified during the session. They answered the questions based on a five-point scale, where the categories from the highest level to the lowest are: (5) clearly adoptable, (4) adoptable, (3) moderately adoptable, (2) little adoptability, (1) not adoptable at all.

Each group of maintainers evaluated one solution generated by the search-based approach, configured to generate five and ten microservices. From the set of solutions returned by the optimization process, we selected the solution with the lowest Euclidean distance to the ideal solution. The ideal solution is the hypothetical one with the lowest value for each fitness function, creating a simulated point with the best fitness values found in the Pareto set.

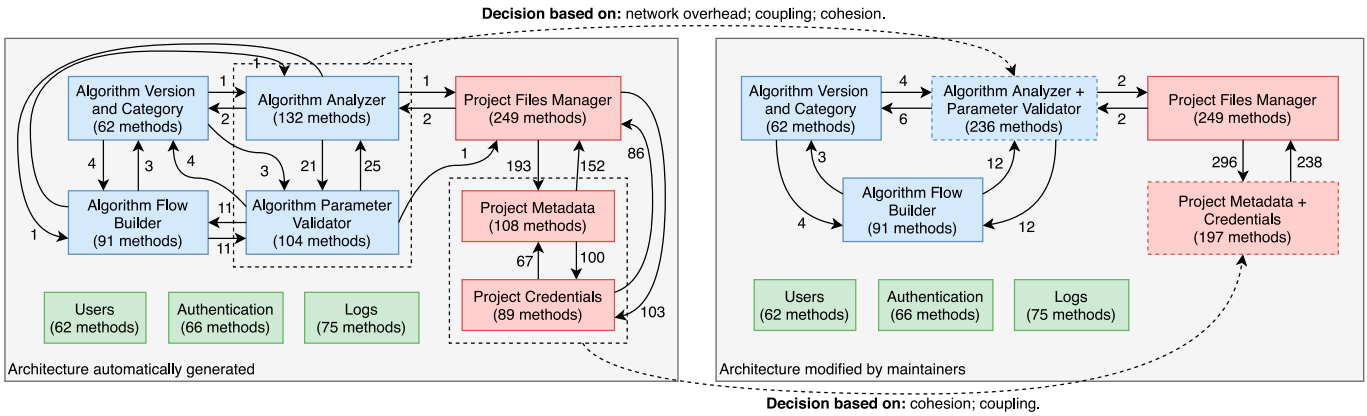
In addition to the seven maintainers, an additional maintainer was selected to perform the refinement operations before the focus group. This maintainer has two years of experience in the maintenance activities of the system under analysis, which includes the knowledge of the features analyzed. The maintainer carried out three complete re-executions, where refinement operations were performed to the solution with the lowest Euclidean distance. He was responsible for performing the refinement operations only to the microservices labeled as adoptable (points 4 and 5 in the Likert scale) in the first study (Section III.C.2). He was not included in the focus group to avoid any bias.

In total, the maintainers accepted 53% of microservices. Then, the additional maintainer spent three hours of re-execution per set (fine-grained and coarse-grained groups) to perform the activities: *(i)* manual analysis of the generated solution, and *(ii)* comparison of feature modularization indicated as positive in the individual microservice evaluation.

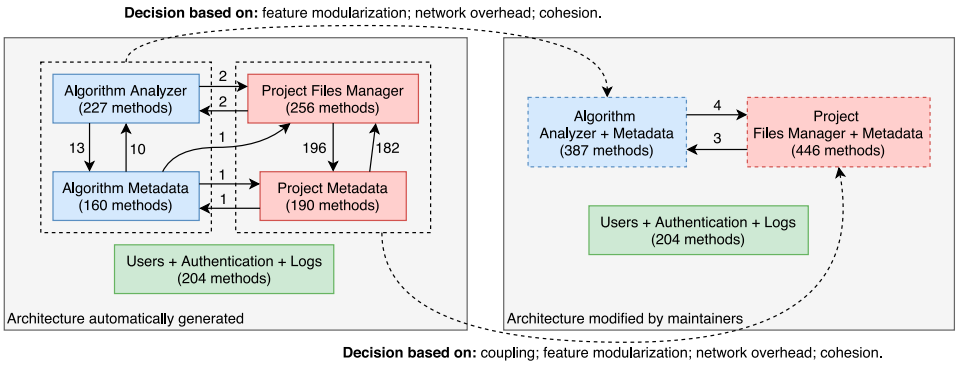
D. Study's Limitations

Our case study is limited to only one legacy system and eight maintainers due to the practical constraints of the industrial setting. Nonetheless, the participants acted in migrating a system they are experienced with to a microservice architecture since

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.



(a) Fine-grained architecture: two design decisions on merging microservices.



(b) Coarse-grained architecture: two design decisions on merging microservices.

Fig. 2. Design decisions by maintainers during focus group for fine- and coarse-grained architectures.

the beginning. Besides, they have been trying to identify microservices for the legacy system before assessing the solutions generated in our study. Furthermore, we needed to deal with time constraints to involve the participants in the microservice's assessment and focus group.

As the existing multi-objective search-based approaches for microservice identification are not entirely aligned with the industrial requirements (see Section III.B.2), we consider that *toMicroservices* is the most representative approach for the set of search-based approaches with the same goal. The research method adopted is appropriate because there are no other equivalent approach to compare the generated solutions (see Section VIII).

According to the nature of case studies, we conducted an in-depth investigation on microservice identification for the subject system, aiming at achieving theoretical generalization by identifying concepts transferable to search-based approaches for microservice identification. In this sense, our findings are significant to improve existing approaches or proposing novel ones, due to the lack of on-site studies to understand the practical adoption and usefulness of automatically generated microservice architectures. Further studies involving other systems can corroborate our findings and enhance the body of knowledge on microservice identification for legacy systems by search-based approaches.

IV. Assessing the Adoptability of Individual Microservices

To answer *RQ1*, we inquired eight participants about the adoptability of microservices, as mentioned in Section III.C.2. Despite the limited number of participants, their specialized opinion is relevant as they are the maintainers engaged in analyzing the alternatives of microservice architecture for the subject system.

Table I presents data about maintainer experience in the legacy system, how many features they identified in the evaluated microservice candidates, and the rates given by participants for the microservice candidates. Notice that the adoptability rate in the last column of Table I does not correspond to the same microservices (see Section III.C.2). The relevant information is regarding the five-point Likert scale, from 1 to 5, where 1 is a microservice the maintainers would not adopt at all, and 5 is a microservice the maintainer would adopt as is.

As seen in Table I, we grouped the participants by scenario based on their choices along with the case study. Participants P1 to P4 were inquired about the adoptability of microservices identified in the first scenario, whereas P5 to P8 for the second scenario. We asked each participant to analyze the adoptability of the four microservices of the corresponding scenario.

Most of the participants are experienced maintainers, with 75% (6 out of 8) having at least two years of experience on the

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

legacy system. Seven participants are maintainers of the legacy system, and one is the team leader. It is an interesting distribution of participants once we can characterize more experienced and less experienced maintainers to analyze the solutions for microservice identification.

RQ1.1. The first analysis was regarding the scope of the microservice candidates. Interestingly, regardless of their experience in the legacy system, all participants could identify the predominant features modularized within the microservices. In addition, the participants also recognized new features (including subfeatures), which are not subfeatures previously informed as known by the participants but identified during the adoptability analysis of the microservice candidates (fourth column of Table I). However, participants with high experience in the legacy system would recognize more features and new features than moderately experienced participants. P5 stated the following with respect to one feature (algorithm) and two subfeatures (parser and writer): *"This microservice is a subset of a more general microservice of algorithm with at least parser and writer"*.

The participants who analyzed solutions of Scenario 1 indicated they would not adopt some microservices in practice because of their large size. According to the participants of this scenario, these microservices had too many methods as part of a coarse-grained feature, and different features undesirably tangled in the same microservice candidate. P2 stated that he would not adopt this microservice because *"It is very large with too many methods, which realize different, unrelated features, i.e., project and authentication"*. The quantitative results indicated that ≈50% of the proposed microservices are partially adoptable or adoptable. P3 argues: *"This microservice realizes management of project files, and it is highly reusable by different tenants."*. Another participant (P1) accepted another microservice: *"The microservice is highly cohesive. However, I would further decompose it into a smaller microservice, even though given certain constraints in our architecture, I would adopt the proposed microservice as is"*.

For Scenario 2, the analysis of the solutions by the maintainers pointed out that they would fully or partially adopt ≈63% of the proposed microservices. The participants indicated three microservices as not feasible to be adopted in practice, mainly because the features modularized by them are too small. P8 said: *"This isolated microservice does not make sense. I believe that it should be merged into the microservice in charge of mastering project files."* However, the same microservice, which modularized the Project Metadata subfeature, was accepted by another participant. Similar contradicting cases occurred among other maintainers. These cases evidence a divergence among participants and their analysis on the adequate level of granularity for the microservice boundaries.

In general, the results regarding the potential adoption of microservices obtained by a search-based approach were: three participants would adopt at least one microservice, four participants would adopt two microservices, and only one participant would not adopt any microservice.

During the interviews, we observed how maintainers reason about the adoptability of microservice candidates. Four maintainers (P1, P2, P6, and P8) presented a profile with a fine-grained microservice perspective, since they positively evaluated microservice candidates that modularized single subfeatures with a very few number of methods. The other three maintainers (P3, P5, and P7) have a coarse-grained microservice profile, where several subfeatures or a complete feature in a microservice was indicated as adequate. The last maintainer (P4) answered that the microservices are not adoptable since they do not present new features or business capabilities.

TABLE II
CRITERIA CITED BY PARTICIPANTS DURING THE MICROSERVICES' ANALYSIS

Participant	# Useful Criteria	Most Influential Criteria
P1	4	Feature Modularization, Reuse, Cohesion
P2	5	Feature Modularization
P3	3	Feature Modularization
P4	7	Feature Modularization
P5	4	Feature Modularization, Cohesion
P6	5	Feature Modularization, Cohesion
P7	5	Feature Modularization
P8	5	Network Overhead, Cohesion, Feature Modularization

We also collected decisions that maintainers performed during this evaluation. These decisions include moving several methods or even removing a single method. For example, P4 stated: *"There are ten methods that should be in another microservice"*. This quote suggests that in the real case, the maintainer would move the methods to another microservice. Hence, the difference in the adoptability was impacted by both the scope of the microservice candidate and the maintainer profile, which shows that the microservice candidates should be in accordance with the maintainer's needs to increase the microservice adoptability.

Finding RQ1.1: *The participants in our study with high experience in a legacy system were able to recognize more features and new features than moderately experienced participants. For Scenario 1, ≈50% of the proposed microservices were found to be partially adoptable or adoptable in practice. For Scenario 2, ≈63% of the proposed microservices were fully or partially adoptable. Participants' profiles highly impact the adoptability of microservices, with fine-grained maintainers positively evaluating microservice candidates that modularized single subfeatures with few methods, while coarse-grained maintainers found several subfeatures or complete features in a microservice to be adequate. Participants made decisions during the evaluation, including moving methods or even removing them to improve adoptability.*

RQ1.2. To answer this RQ, we identified how many criteria the participants analyzed and considered useful. The number of criteria identified for each participant is presented in the second column of Table II. One participant reasoned about seven criteria. Four participant considered five criteria. Two participants took into account four criteria during their analysis. In one case, a participant considered only three criteria during the analysis of solutions. These results are consonant with the

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

TABLE III
RANKING OF CRITERIA

Criteria	Rank	Criteria	Rank
Feature Modularization	8	Business	2
Cohesion	8	Performance	1
Coupling	8	Security	1
Communication Overhead	3	Maintainability	1
Reuse	2	Interface Complexity	1
Team Size	2	Maintainer Knowledge	1

literature reports, which state that in practice many criteria are useful during the identification of microservices [12], [13].

The last column of Table II presents the most mentioned criteria when considering microservices' characteristics. We can say that these are the most "preferable" criteria from the point of view of maintainers along with our case study. We can observe in the column that feature modularization is the main criterion used in the analysis. It corroborates with the analysis in the previous research question, where the participants many times reasoned about features and subfeatures to confirm or refute the adoption of a microservice candidate. We can infer that maintainers search for microservices containing well-defined, cohesive, and reusable (sub)features. Cohesion is also often mentioned. Reuse and network overhead were taken into account as the favorite criteria only by two maintainers. We believe these criteria are more difficult to analyze, and the maintainers would need more time or adequate tools to reason about them.

Based on the information collected during the interviews, we produced a ranking of the criteria mentioned by participants during the microservice evaluation (see Table III). The second and the fourth columns of Table III present the number of times each criterion was mentioned. Feature modularization, cohesion, and coupling were taken into account by all participants. Interestingly, the five criteria optimized by *toMicroservices* are the most cited by participants, even not knowing which criteria are adopted by the approach.

Given the specifics of the case (see Section III.B.1), technical criteria were more relevant. Nonetheless, some participants also mentioned non-technical criteria, such as business, team size, and maintainer knowledge. In addition, criteria related to quality requirements were mentioned once, namely performance, security, maintainability, and interface complexity.

Finding RQ1.2: *The findings highlight that professionals should consider multiple criteria when evaluating microservices to ensure their effectiveness and suitability for adoption.*

V. ADOPTABILITY OF MICROSERVICE ARCHITECTURES

In this section, we report the results of the focus group conducted to assess the adoptability of the microservice architecture generated by the search-based approach. The focus group study led us to observe global architectural decisions within the context of the industrial legacy system presented in Section III.B.1. The analyses provided during the individual microservice evaluation (Section IV) were used during an interactive process with the search-based approach to refine the solutions.

TABLE IV
ADOPTABILITY OF THE MICROSERVICE ARCHITECTURES

Architecture	Median Rate	
	Fine-Grained	Coarse-Grained
Initial architecture	4, 3, 3, 2	1, 1, 2
Final architecture	5, 4, 4, 4	4, 4, 3

A. Focus Group Artifacts

Fig. 2 presents the visual representation of the microservice architecture evaluated in the focus group. Regarding the fine-grained architecture (left side of Fig. 2(a)), the Project Credentials microservice is responsible for providing information on who are the users who have permissions on the projects, for example, the project owner and project members. Project Metadata is responsible for information and operation in metadata as project name, type, and description. Finally, the Project Files Manager contains files associated with each project and manages downloads and uploads. Regarding the algorithm feature, the Algorithm Analyzer is responsible for creating algorithms, including modifier parameters, versions, and documents. Algorithm Version and Category manage the different versions of each algorithm, including differences as input parameters that should be provided in a specific format to execute them. The Algorithm Flow Builder is responsible for composing a workflow of the algorithm. That is, the plan sequence of algorithm execution, where the algorithms in the flow can be the input and output linked. The Algorithm Parameter Validator stores the parameters of each algorithm existent and can validate a real instance of an algorithm's input and output (*e.g.*, the request to execute an algorithm).

The left side of Fig. 2(b) shows the microservice architecture generated by the search-based approach, configured to generate five microservices. In this solution, the Algorithm Analyzer configures and validates the parameters of each algorithm. Still, regarding the algorithm feature, Algorithm Metadata is responsible for the subfeature of algorithm flow, versions, and categories. In summary, the Project File Manager is concerned about the file system, and Project Metadata is concerned about information such as owner, users per shared file, and additional credentials per project. Given the number of desired microservices, only one microservice is responsible for features related to user, authentication, and logs in the coarse-grained solution.

B. Results

In this section, we present the focus group results in order to answer the RQ2 and their sub-research questions.

RQ2.1. The focus group moderator collected the modifications suggested during the study execution. The goal was to understand which modifications were carried out to make the generated solutions adoptable for the legacy system. In addition, the maintainers were inquired whether the architecture generated by the automated approach was adoptable. Table IV shows the result in terms of the median of a five-point Likert scale. The maintainers of the *fine-grained* group indicated a moderate adoptability of the initial solution. The maintainers of

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

the *coarse-grained* group pointed out that the initial architecture was not adoptable.

The maintainers were also inquired about the adoptability of the modified solution during the focus group session (final architecture). The median equals 4 suggests the adoptability of the solution for both fine-grained and coarse-grained groups (see Section III.C.4). To answer ***RQ2.1***, the analysis of both presented questions suggested that the maintainers would not adopt the solution generated by the search-based approach. However, with modifications such as those performed in the focus group sessions, the solution would be adopted for the legacy system.

Regarding methods in each microservice, the maintainers of the *fine-grained* group moved less than 1% of the methods to another microservice. The methods moved are related to the authentication feature. In the *coarse-grained* group, no method was moved. The additional modifications performed by both groups were limited to merging pairs of microservices. In response to ***RQ2.1***, the modifications that mainly consist of merging microservices pairs lead to a low effort to modify the automatically generated microservice architecture. In summary, this fact shows the suitability of the microservices and their related code, motivated by the addition of interaction with an expert in the legacy system. Regarding the ability to recognize features and subfeatures, all maintainers were able to recognize them through visual representation and the source code presented. In addition, additional subfeatures were not indicated by maintainers during the analysis of the architectures, nor subfeatures were pointed as poorly modularized in the microservices.

Finding RQ2.1: *The maintainers of the fine-grained group indicated a moderate adoptability of the initial solution, while the maintainers of the coarse-grained group pointed out that the initial architecture was not adoptable. The analysis of the questions suggested that maintainers would not adopt the solution generated by the search-based approach without modifications. Regarding the methods in each microservice, both groups made limited modifications, predominantly merging pairs of microservices.*

RQ2.2. The architecture presented in Fig. 2 was evaluated by the group of maintainers with the *fine-grained* and *coarse-grained* profiles.

Fine-grained group. In the first moment, P8 said about the Algorithm Version and Category: *"Why is the version subfeature in this microservice?"*. The participant also argued that: *"version subfeature should be present in the Algorithm Analyzer microservice"*. He observes an intrinsic potential relationship. However, another participant (P1) stated: *"Version and category are reusable and cohesive in this microservice"*. The maintainers also pondered that a new version leads to a new parameter validator (modularized in the Algorithm Parameter Validator). Thus, this case makes the Version and Catalog microservice necessary and unmerged from the Algorithm Analyzer microservice to provide traceability.

In a general way, P6 said: *'The architecture has a high microservice decomposition. It adds little disadvantage to the system. For example, the architecture can cause a prohibitive overhead between microservices of the algorithm feature."* In a continuation stance, he pointed out that all the cases requested to the interface of the Algorithm Analyzer microservice will necessarily pass by the Algorithm Parameter Validator microservice. In this order, pointing to high coupling as visualized in Fig. 2(a), and high network overhead since the transmitted data can be high. Another participant (P2) added that: *"Both microservices will contain many states with several pieces of information."* All the maintainers agree to merge the Algorithm Analyzer and Algorithm Parameter microservices as shown on the right side of Fig. 2(a), although the other criteria are positive (feature modularization and reuse) as pointed by them.

For the Algorithm Parameter microservice, a maintainer points out that the reuse of this microservice is high. For this instance, the maintainer argues that this microservice could be modified by others to comply with the requirements of other customers. Thus, the microservices improve system customization for future evolutions. Regarding Algorithm Flow Builder microservice, P6 argued: *"Flow in a unique microservice is good as it can be easily handled in different ways"*. This quote points to the high reuse of the microservice.

When discussing the project feature, all four maintainers agreed that Project Files Manager is an adoptable microservice since modularizing this subfeature in a microservice allows dealing better with variability in the future [13], [44], for instance, if some customer requires changing a traditional file system to a distributed file system. Regarding the relationship of Project Files Manager with the other two microservices of the project subfeatures, a participant first stated that performance is a potential problem since each file requested will necessarily generate a request from the Project Files Manager microservice to the Project Credentials microservice. Thus, the participant pondered a possible high network overhead. In the same way, another participant said that, in this case, it is possible to make a cache since the transferred information has low size and changes rarely. Moreover, the participant pointed out that it is possible to perform a data duplication. Thus, network overhead was not considered a problem in the relationship between Project Credentials and Project Metadata since the possible data analyzed is low and cacheable.

Regarding the Project Credentials and Project Metadata microservices, the maintainers observed that would be more cohesive if they are merged. They observed high coupling between them, as depicted in Fig. 2(a), in spite of implementing different features and having some level of data separation. For instance, the project name and date of creation in Project Metadata, and permission to access a specific file in Project Credentials. However, particular cases generated discussions. For example, the participants discuss where the information of the project owner can be contained. They agree that the Project Metadata microservice or Project Credentials microservice should contain or share the data. Thus, they consider cohesion and coupling sufficiently violated and decided to merge these microservices. Finally, maintainers considered adoptable the Project Files Manager and Project Credentials + Metadata microservices.

Coarse-grained group. This group was even more restricted with the number of microservices, as shown in Fig. 2(b). In the

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

first moment, during the analyses of the proposed microservice architecture, P5 said *"There is no sense having a microservice of metadata to project and algorithm. Must have only the project and algorithm microservices. If not, we will get microservices with little subfeatures"*. P3 observed that: *"The logic to identify the microservices is the same: one microservice to store and another modularizing the main feature"*. P3 agrees with the first statement. The *fine-grained* microservices can generate a high network overhead in Project Metadata.

The P5 continued to criticize the proposed microservice architecture: *"The communications between microservices are very high. This is a strong signal to merge the microservices"*. After that, the same maintainer illustrated a bad decision: *"182 communications between these two microservices indicate a high level of coupling"*. In this order, the maintainer considered that the granularity of the feature is enough for this case, and defended that the microservice architecture should be composed of one microservice for the feature Project and another microservice for the feature Algorithm.

Two maintainers (P3 and P5) understood Algorithm Metadata and Project Files Manager as a layer analogous to a model-view-controller architecture, and not as microservices. P7 complemented: *"Microservices need to have a responsibility well-defined. The goal of the microservice must be well-defined. In this case, the information should be structured in an internal way as a component and not a microservice"*. Thus, this group of maintainers decided to merge the microservices related to the subfeatures into a unique microservice with to increase feature modularization and cohesion and minimize network overhead, as may be seen on the right side of Fig. 2(b).

The criteria used by the maintainers to argue the adoption of the microservice architecture and the adaptations were coded by the authors. Our results point out that all criteria optimized by the automated approach were used in practice by the maintainers. For instance, P6 stated: *"Version and algorithm catalog are reusable microservice. Also, there is cohesion"*. In our coding analysis, the codes of cohesion and reuse criteria were assigned to this statement.

Other criteria were not presented in a direct way. Among them, one P6 said: *"It is always necessary to pass by the microservice of the algorithm version, once that algorithm data are requested"*. The statement does not contain the coupling word. However, the high degree of dependency between the Algorithm version and an additional microservice in the algorithm feature points to a high undesired degree of coupling. Thus, our coding points to the use of the coupling criterion in the aforementioned phrase.

For the feature modularization criterion, P1 inquired the other maintainers whether permission of algorithm and project should be modularized by a single microservice: *"Must permissions be all together?"*. The other maintainers stated that it would lead to several united information and responsibilities. Moreover, several cases were described by the maintainers to justify the isolation of both subfeatures, such as, cases in which a user has permission to access the project file and does not manage an algorithm or execute it.

Coupling criteria were used by P5, which has a coarse-grained profile, to argue the merge of two microservices. He stated: *"182 communications between these two microservices indicate a high level of coupling"*. The additional maintainers (P3 and P7) agreed with the statement and decided to merge both microservices. Fig. 2(a) and 2(b) present the merge motives, under the point of view of criteria, to each pair of microservices merged.

Finding RQ2.2: *We highlight that no method was moved between microservices, which allows inferring that the refinement operations are effective to achieve better solutions. All criteria used by the automated approach have been used to analyze the relationship between different microservices and the entire architecture. The qualitative analyses show that all criteria are relevant and that none criterion is always predominant from others.*

RQ2.3. Considering the adaptations and discussion of *fine* and *coarse-grained* groups, it is clear that the profile of maintainers influences the design decisions. For the same legacy system, both groups took different decisions prioritizing different criteria as discussed in the answer for RQ2.2. In general, the fine-grained group did not prioritize the feature modularization criteria, since the microservices were already well-modularized in terms of subfeatures. They were concerned with the network overhead, coupling, cohesion, and the potential for microservice reuse or customization.

Different from the fine-grained group, the coarse-grained group did not cite directly the reuse criterion or any other description associated with reuse. This observation indicates that the maintainers with coarse-grained profiles did not consider or observe the possibility of microservice reuse. Thus, they only modularized microservices in the level of features (project and algorithm) and with more methods than the fine-grained group. Network overhead and cohesion were also important for this group.

Finding RQ2.3: *The study showed that the profile of maintainers influences design decisions, as the fine-grained and coarse-grained groups prioritized different criteria when making adaptations to the generated microservice architecture. The fine-grained group focused on network overhead, coupling, cohesion, and potential for microservice reuse or customization, while the coarse-grained group did not consider microservice reuse and instead focused on feature modularization, with more methods in each microservice. Both groups valued network overhead and cohesion. These findings highlight the importance of considering the profile of maintainers when designing microservice architectures. Moreover, it suggests more studies need to be performed in this field in order to consider other systems, contexts, and more participants.*

VI. Lessons Learned

Migrating to microservices is a complex process that requires careful planning and execution. The results of our case study allowed us to learn relevant lessons, which represent identified concepts that are transferable and can be used to propose novel approaches. For each lesson learned, we also suggest how

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

to improve existing search-based approaches to microservice identification. We present below six future directions that could advance the field of microservices migration:

1. Automated approaches should take into account the maintainers' preferences. Based on the results, we observed that microservices identified by a search-based approach would be adopted by maintainers. However, some adaptation cases were pointed out depending on maintainers' preferences. Based on that, we could learn that the profiles of maintainers influence the decision-making process regarding the granularity of microservice, and consequently the characteristics of the microservice architectures.[2] This lesson opens opportunities for further improvements of state-of-the-art search-based approaches.

There is a need for a maintainer-centered design approach that takes into account maintainer feedback in the migration process. Such an approach should consider the unique needs of each maintainer group (based on their profile similarity) and ensure that the microservices are designed to meet those needs. Thus, solutions can be created with more adequacy for different profiles and different contexts. This is motivated by the results of RQs 2.2 and 2.3, which show that maintainers prioritize different criteria depending on their profile. Existing approaches can be improved to allow the users feedback about their criteria and preferences before the search process is performed. For instance, asking for the maintainers' preference for criteria first, then optimizing the corresponding criteria. Also, there is an opportunity to develop methods for analyzing user feedback to identify patterns and trends that can cluster profiles and recommend microservices. This research can explore the use of natural language processing and machine learning techniques. Such feedback can be assessing the quality of microservices or the entire architecture, assigning a preference rate. Additionally, a search-based approach can implement operations for splitting/merging microservices, moving methods between microservices, or keeping some parts of the architecture unchanged. Finally, we highlight the importance of conducting controlled experimentation to compare the solution's adoptability, by considering: *(i)* the use of some general criteria and preferences, and *(ii)* the use of specific criteria and preferences defined by maintainers.

2. Maintainers reason about the microservices and the architecture in different granularity levels and perspectives. As discussed in the answers of RQs 1.1 and 2.2, decisions made by maintainers include moving several methods across microservices or even removing a single method from a microservice. These decisions lead to the proposal of two refinement operations to move or freeze methods. We also observed in the qualitative analysis of common maintainers' decisions (Section V, results of *RQ2.2*) that they merge some microservices into a single microservice. In summary, maintainers reason about microservices and architectures from different levels of granularity and scopes, depending on their role and responsibilities within the organization. We learned that using the abstraction level of methods enables maintainers to have more flexibility when reasoning about microservices. However, most existing approaches are restrictive, as they usually perform a coarse-grained search at the level of modules (or files), hindering the design decisions at a fine-grained level (method level). Thus, for practical adoption, approaches must allow maintainers to decide which abstraction level to work on, enabling them to reason about microservices from different perspectives and helping to ensure that the microservice architecture is designed, implemented, and maintained effectively.

3. The solutions automatically obtained by the search-based approach are a good starting point for microservice identification. From the solutions obtained automatically, only a few local improvements were proposed during design decisions made in the Microservice Architecture Evaluation (Section V). This is also an important achievement, as performing the microservice identification from the legacy system manually is a high-demand and error-prone activity. Thus, we have learned that despite search-based approaches not reaching optimal solutions, the obtained architectures were good alleviating the burden of maintainers and speeding up the modernization process. Additionally, the solutions automatically obtained helped maintainers to learn more about the legacy system, as the identification of subfeatures (discussed in RQ 1.1). Thus, it is important to notice that traditional metrics may not be enough to measure the user experience with the approach. In this context, as future work, researchers can investigate the development of new metrics that focus on user experience in a microservice context.

4. Scattered features need further attention. Scattered features refer to features that have their implementation spread across multiple artifacts (*e.g.*, classes). We observed that grouping the implementation of scattered features in a single microservice is challenging. For instance, Authentication was not captured by a microservice as a main feature, as discussed in the results of RQ 2.1. The Authentication methods were scattered between several microservices in the scenarios with 5 and 10 microservices. The scattered methods of Authentication can be due to two reasons: *(i)* bad modularization in the legacy system, and *(ii)* the cross-cutting nature of the feature. An intensive bad modularization should be difficult to convert to acceptable values of cohesion and coupling in a microservice. In addition, the crosscutting feature may be optimized with different fitness values. In future works, empirical studies with other legacy systems should be carried out with the goal of understanding how scattered features are modularized and how to deal with crosscutting features during microservice identification. Further studies can guide the proposal of new fitness functions to be used in search-based approaches to deal with these kinds of features. Identifying these features and determining how to handle them effectively is critical to ensuring the scalability, maintainability, and overall success of the microservice architectures.

5. The visualization of microservice architectures is important to maintainers. Despite not being the focus of our study, we observed that the visualization of artifacts is relevant, as already investigated in the literature [45]. The maintainers

[2]We observed only different granularity preferences. Despite we generalized the rationale for the profile, in other situations, other kinds of preferences can emerge in addition to the granularity ones.

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

recommended in both the individual microservice evaluation (RQ 1) and in the microservice architecture evaluation (RQ 2): *(i)* to remove auxiliary functions providing a "clear" view of the microservices, *(ii)* to remove dependencies that will not exist in the migrated system, and *(iii)* to highlight the predominant feature or subfeature in each microservice. The maintainers also pointed out that the visualization of microservices based on dynamic analysis is important. In this order, they pointed to the frequency at which each microservice was called by a dynamic perspective as relevant. Moreover, the maintainers stated the importance of presenting *(i)* the data flow between microservices as additional content to the interfaces and relationship, and *(ii)* the frequency, based on dynamic information. Another suggestion directly presented by maintainers was concerned with the visualization of the content of each microservice from the perspective of modules or components. That is, the visualization could present the architecture of each microservice. For example, a microservice with a layered architecture should be each layer and this relationship presented. In the case of the use of domain-driven design, the visualization could present the entities and internal bound context. Based on this observation, we learned that new approaches should be concerned with the visualization of the static and dynamic characteristics of the legacy and the microservices identified from different perspectives/granularity. Visualization can help ensure the overall success of the microservice architecture.

6. Non-technical criteria and criteria related to quality requirements are also taken into account by some maintainers. The participants of our study reasoned about quality requirements, such as performance, security, and interface complexity, during the evaluation of microservices, as can be observed in the results of RQ 1.2 (Table III). In other cases, managerial aspects, such as independent deployment, full-lifecycle management, and deployment by a small autonomous team, can also be relevant. In this sense, search-based approaches can be extended to include fitness functions that evaluate such kind of criteria. On the other hand, it is not so easy to include fitness functions to evaluate the non-technical criteria mentioned by the maintainers (*e.g.*, team size, business, and maintainer knowledge). However, they might be considered by the maintainers when evaluating a solution by means of user interaction during the optimization process, as discussed in the first lesson learned.

VII. Threats to Validity

This section discusses potential threats to the validity of our work and the strategies we have followed to minimize their impact. We discussed the study validity with respect to the two groups of common threats to validity: *internal* and *external* validity [46].

Internal validity. A major internal validity threat concerns the set of microservices analyzed by maintainers of the legacy system. Our automated approach generates a set of solutions, *i.e.*, a set of microservice architectures with their correspondent set of all possible microservice candidates and their interdependencies in each solution; however, it is very exhaustive and time-consuming for a maintainer to evaluate every solution. To mitigate this threat, we inquired each maintainer on the adoptability of a set of microservices selected by the same microservice architecture.

For the interview, we execute the data analysis process using the open-coding method, which is a subjective task. Thus, a potential threat to validity is the reliability and accuracy of the coding process, since not all information was obvious to extract (*e.g.*, the divergence of researchers' opinions about which criteria were taken into account by each maintainer). Consequently, we needed to interpret the subjective data addressing the researchers' decisions. Therefore, to avoid bias, we carefully involve three distinct authors in coding, refinements, validation, and discussions. Still, to further ensure the validity, a fourth author cross-checked and discussed the outcome. Moreover, whenever there were disagreements, we involved a fifth author. We discussed the coding from different perspectives until a final decision was achieved, resolving all discrepancies to converge and decide on the adopted code. Furthermore, we documented all potentially relevant discussions.

For the focus group, a potential threat to validity is related to how we designed the protocol. To mitigate potential problems in the study design, we have performed a pilot execution of our initial protocol with a group of two maintainers of the system under analysis. Moreover, we also involved a specialist researcher to review the generated artifacts. Based on their feedback, we have identified problems and improved significantly the protocol. We mainly created *(i)* refinements on the visual representation to better the position of the edges and their weight, and *(ii)* a summary of the subfeatures in each microservice.

External validity. There is an inherent threat to external validity in generalizing our findings for other legacy systems The challenges of conducting an empirical study in an industrial setting, such as ours, are significant. However, mitigating this threat is not straightforward due to the lack of access to other real-world legacy systems, and the difficulty of engaging maintainers of these systems for experiments, surveys, and interviews. The practical constraints in an industrial setting impose limitations on the scale of the study, such as the availability of practitioners constrained by their demanding work schedules and other priorities. Thus, we mitigate this threat by selecting a consolidated real-world legacy system with more than 15 years of existence and under the process of migration to microservices. We focused on a single system and eight maintainers to be able to make a robust and reliable empirical study. The goal of our study, as per ACM SIGSOFT Empirical Standards,[3] is to achieve theoretical generalization by identifying concepts that are transferable in principle. Once we demonstrate evidence to the considered system, we plan to search for new business partnerships to perform such an analysis over other systems and more participants and thus generalize our findings.

Another external threat to validity is the different levels of experience of maintainers interviewed in our study. We tried

[3]https://www2.sigsoft.org/EmpiricalStandards/about/

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

to minimize this threat by involving mostly experienced maintainers who were knowledgeable about the legacy system. Also, we have performed a training session to align their knowledge about the search-based approach we used to identify microservices (*toMicroservices*).

A limitation of our study is that we analyzed the maintainer profile only from the perspective of the microservice granularity preferences. As other preferences can emerge in different situations, conducting further studies to investigate the profile from other perspectives is essential.

Another threat to validity concerns the discussion of the results and lessons learned. To avoid bias, we have conducted a discussion with all authors. We are aware that it is also important to explore other approaches beyond *toMicroservices* to fully gather the spread knowledge, which may extend the list of findings in this field. However, there is a lack of a meaningful comparison baseline (see Section VIII).

VIII. Related Work

A wide range of studies has been reporting the process of migrating to the microservice architecture [10], [16], [32]. These studies commonly report the motivations, benefits, and challenges associated with the migration process.

Fowler [6] reported the migration of Uber API into a monolithic system and afterward in a microservice architecture. The migration resulted in more than 1,000 microservices extracted from the Uber monolithic system. Similarly, Bucchiarone et al. [9] also presented an experience report, but in the context of a mission-critical system in the banking domain. Luz et al. [4] reported the experience of migrating legacy systems to microservice architecture at three Brazilian government institutions. Among the limitations of the legacy systems, choosing technologies imposed by the legacy system architecture, high coupling with shared database, not independent deployment process, and the like. After the migration processes, the maintainers perceived several benefits, including better continuous delivery, deployment more flexible, leverage of innovation, reduction in time to market, enable of DevOps adoption, and increased availability.

Additional studies [3], [8], [47] also report other experiences on migrating legacy systems to a microservice architecture, similarly to the above studies. However, these studies did not present an in-depth analysis of the decision-making process to evaluate how adequate is a microservice in the context of migration. Among them, it is missing an empirical evaluation with the maintainers to assess the adequacy of the generated microservices.

Taibi et al. [5] conducted a survey with the objective of elucidating motivations that led maintainers to the process of migrating to a microservice architecture and what were the expected benefits. Francesco et al. [15] interviewed and applied a questionnaire to experienced maintainers. Their goal was to understand the characteristics of the migrating process and the challenges (*e.g.*, difficulty of identifying the service boundaries) faced during the migration. The more common artifacts are the source code and documents. Moreover, the main reported challenges were: *(i)* the high level of coupling, *(ii)* the difficulty of identifying the service boundaries, and *(iii)* the microservice decomposition.

Carvalho et al. [12], [13] inquired experienced maintainers about the usefulness of criteria to identify microservices that relied on legacy code. The results point out useful criteria for extracting microservices and that previous automated approaches oversimplified microservice identification. In this way, we used these criteria to build the *toMicroservices* approach [21]. However, similar to the previous related works, our previous study did not perform an in-depth analysis of the decision-making process followed by maintainers reasoning about individual microservices and microservice architectures.

Some studies focus on using search-based approaches to identify microservices, considering different aspects [16]. Saidani et al. [22] propose the use of multi-objective optimization in a tool called MSExtractor. However, their focus was uniquely on coupling and cohesion using class-level analysis, and the evaluation was based on an open-source system. Zhang et al. [23] consider both functional and non-functional metrics, extracting information from execution and performance logs. Then, a multi-objective optimization performs clustering of classes to produce microservices with high-cohesion-low-coupling and load balance. The work of Christoforou et al. [24] focuses on functional properties, with a process built on semi-formal profiling and utilizing ontologies. Jin et al. [18] present a framework for service candidate identification based on execution traces. The execution traces of the monolith system are recorded and used as input to a multi-objective optimization. The services are evaluated regarding functionality, modularity, and evolvability. All these pieces of work above mostly evaluate the microservices individually in terms of metrics and performance measures, without interviewing maintainers about the entire architecture, which is the goal of our work.

Khoshnevis [26] propose SAMIM, a method for identifying microservices based on multi-objective optimization. The SAMIM's main goal is to search for microservices that are based on business processes, producing a microservice architecture that is reusable, consistent, and variable. Differently from our work, Khoshnevis focused on evaluating the optimization method using as baseline expected architectures produced by experts. Thus, Khoshnevis can be seen as complementary to our work.

IX. Conclusion

In this paper, we conducted an industrial case study to investigate the challenges of identifying microservices from legacy code. Our results confirm that the automated approach is able to identify adoptable microservices from the perspective of maintainers. Observing the maintainers' assessment led us to refine the search-based approach to include the maintainers' feedback and allow user interaction between the optimization process and the maintainers' assessment. The results also revealed the influence of the maintainers' profiles on assessing the microservices' adoptability for composing a final microservice architecture. Furthermore, we presented insights into

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

typical discussions of maintainers when evaluating a proposed microservice architecture.

Our study indicated that the automatically generated microservice architectures are a good starting point and that maintainers need to perform few simple changes to achieve an adoptable architecture, which is better than performing the whole process manually. Another important finding is that the identification of microservices from legacy systems is highly dependent on the maintainers' preferences, thus adaptive search-based approaches are capable of generating more useful microservice architecture solutions.

For further studies, we plan to search for new business partnerships to demonstrate the generalization of our findings for other legacy systems, contexts, and maintainers. Moreover, an interesting direction is to investigate the possible impact of the automatically generated microservice architectures on the future changes of the system. Furthermore, exploring how to prevent microservice anti-patterns since the microservice migration process can lead to interesting results.

Acknowledgment

The authors thank Tecgraf Institute of PUC-Rio for its support in the case study.

References

[1] J. Bisbal, D. Lawless, B. Wu, and J. Grimson, "Legacy information systems: Issues and directions," *IEEE Softw.*, vol. 16, no. 5, pp. 103–111, Sep. 1999.

[2] J. Ransom, I. Somerville, and I. Warren, "A method for assessing legacy systems for evolution," in *Proc. 2nd Euromicro Conf. Softw. Maintenance Reeng.*, Mar. 1998, pp. 128–134.

[3] L. P. Tizzei, M. Nery, V. C. V. B. Segura, and R. Cerqueira, "Using microservices and software product line engineering to support reuse of evolving multi-tenant SaaS," in *Proc. 21st Int. Syst. Softw. Product Line Conf. (SPLC)*, 2017, pp. 205–214.

[4] W. Luz, E. Agilar, M. C. de Oliveira, C. E. R. de Melo, G. Pinto, and R. Bonifácio, "An experience report on the adoption of microservices in three Brazilian government institutions," in *Proc. XXXII Brazilian Symp. Softw. Eng. (SBES)*, New York, NY, USA: ACM, 2018, pp. 32–41.

[5] D. Taibi, V. Lenarduzzi, and C. Pahl, "Processes, motivations, and issues for migrating to microservices architectures: An empirical investigation," *IEEE Cloud Comput.*, vol. 4, no. 5, pp. 22–32, Sep./Oct. 2017.

[6] S. Fowler, *Production-Ready Microservices*. Sebastopol, CA, USA: O'Reilly Media, 2016.

[7] C. Watson, S. Emmons, and B. Gregg, "A microscope on microservices," Netflix, 2015. Accessed: Jul. 12, 2022. [Online]. Available: http://techblog.netflix.com/2015/02/a-microscope-on-microservices.html

[8] J. Gouigoux and D. Tamzalit, "From monolith to microservices: Lessons learned on an industrial migration to a web oriented architecture," in *Proc. Int. Conf. Softw. Archit. Workshops (ICSAW)*, 2017, pp. 62–65.

[9] A. Bucchiarone, N. Dragoni, S. Dustdar, S. T. Larsen, and M. Mazzara, "From monolithic to microservices: An experience report from the banking domain," *IEEE Softw.*, vol. 35, no. 3, pp. 50–55, May/Jun. 2018.

[10] D. Wolfart et al., "Modernizing legacy systems with microservices: A roadmap," in *Proc. 25th Eval. Assessment Softw. Eng.*, New York, NY, USA: ACM, 2021, pp. 149–159.

[11] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc., 2015.

[12] L. Carvalho, A. Garcia, W. K. G. Assuncão, R. de Mello, and M. J. de Lima, "Analysis of the criteria adopted in industry to extract microservices," in *Proc. 7th Int. Workshop Conducting Empirical Stud. Ind. 6th Int. Workshop Softw. Eng. Res. Ind. Pract.*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 22–29.

[13] L. Carvalho, A. Garcia, W. K. G. Assuncão, R. Bonifácio, L. P. Tizzei, and T. E. Colanzi, "Extraction of configurable and reusable microservices from legacy systems: An exploratory study," in *Proc. 23rd Int. Syst. Softw. Product Line Conf. Vol. A (SPLC)*, New York, NY, USA: ACM, 2019, pp. 26–31.

[14] W. K. Assuncão et al., "A multi-criteria strategy for redesigning legacy features as microservices: An industrial case study," in *Proc. IEEE Int. Conf. Softw. Anal., Evolution Reeng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 377–387.

[15] P. D. Francesco, P. Lago, and I. Malavolta, "Migrating towards microservice architectures: An industrial survey," in *Proc. Int. Conf. Softw. Archit. (ICSA)*, 2018, pp. 29:01–29:09.

[16] Y. Abgaz et al., "Decomposition of monolith applications into microservices architectures: A systematic review," *IEEE Trans. Softw. Eng.*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023.

[17] W. Jin, T. Liu, Q. Zheng, D. Cui, and Y. Cai, "Functionality-oriented microservice extraction based on execution trace clustering," in *Proc. Int. Conf. Web Services (ICWS)*, 2018, pp. 211–218.

[18] W. Jin, T. Liu, Y. Cai, R. Kazman, R. Mo, and Q. Zheng, "Service candidate identification from monolithic systems based on execution traces," *IEEE Trans. Softw. Eng.*, vol. 47, no. 5, pp. 987–1007, May 2021.

[19] A. K. Kalia et al., "Mono2Micro: An AI-based toolchain for evolving monolithic enterprise applications to a microservice architecture," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 1606–1610.

[20] Y. Zhang, B. Liu, L. Dai, K. Chen, and X. Cao, "Automated microservice identification in legacy systems with functional and non-functional metrics," in *Proc. IEEE Int. Conf. Softw. Archit. (ICSA)*, Mar. 2020, pp. 135–145.

[21] L. Carvalho et al., "On the performance and adoption of search-based microservice identification with toMicroservices," in *Proc. 36th IEEE Int. Conf. Softw. Maintenance Evolution (ICSME)*, 2020, pp. 569–580.

[22] I. Saidani, A. Ouni, M. W. Mkaouer, and A. Saied, "Towards automated microservices extraction using multi-objective evolutionary search," in *Proc. 17th Int. Conf. Service-Oriented Comput.*, New York, NY, USA: Springer-Verlag, 2019, pp. 58–63.

[23] Y. Zhang, B. Liu, L. Dai, K. Chen, and X. Cao, "Automated microservice identification in legacy systems with functional and non-functional metrics," in *Proc. IEEE Int. Conf. Softw. Archit. (ICSA)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 135–145.

[24] A. Christoforou, L. Odysseos, and A. Andreou, "Migration of software components to microservices: Matching and synthesis," in *Proc. 14th Int. Conf. Eval. Novel Approaches Softw. Eng.*, Setúbal, Portugal: SciTePress, 2019, pp. 134–146.

[25] S. Eski and F. Buzluca, "An automatic extraction approach: Transition to microservices architecture from monolithic application," in *Proc. 19th Int. Conf. Agile Softw. Develop.: Companion*, New York, NY, USA: ACM, 2018, pp. 25: 1–25:6.

[26] S. Khoshnevis, "A search-based identification of variable microservices for enterprise SaaS," *Frontiers Comput. Sci.*, vol. 17, no. 3, 2023, Art. no. 173208.

[27] G. Mazlami, J. Cito, and P. Leitner, "Extraction of microservices from monolithic software architectures," in *Proc. Int. Conf. Web Services (ICWS)*, Piscataway, NJ, USA: IEEE Press, 2017, pp. 524–531.

[28] D. Escobar et al., "Towards the understanding and evolution of monolithic applications as microservices," in *Proc. XLII Latin Amer. Comput. Conf. (CLEI)*, Piscataway, NJ, USA: IEEE Press, 2016, pp. 1–11.

[29] J. Lewis and M. Fowler, "Microservices," martinFowler.com, 2014. Accessed: Jul. 12, 2022. [Online]. Available: https://martinfowler.com/articles/microservices.html

[30] J. Kontio, J. Bragge, and L. Lehtola, *The Focus Group Method as an Empirical Tool in Software Engineering*. New York, NY, USA: Springer-Verlag, 2008, pp. 93–116.

[31] W. K. Assuncão, J. Krüger, S. Mosser, and S. Selaoui, "How do microservices evolve? An empirical analysis of changes in open-source microservice repositories," *J. Syst. Softw.*, vol. 204, Oct. 2023, Art. no. 111788.

[32] P. Di Francesco, P. Lago, and I. Malavolta, "Architecting with microservices: A systematic mapping study," *J. Syst. Softw.*, vol. 150, pp. 77–97, Apr. 2019.

[33] T. Colanzi et al., "Are we speaking the industry language? The practice and literature of modernizing legacy systems with microservices," in *Proc. 15th Brazilian Symp. Softw. Compon., Architect., Reuse*, 2021, pp. 61–70.

[34] M. Harman and B. F. Jones, "Search-based software engineering," *Inf. Softw. Technol.*, vol. 43, no. 14, pp. 833–839, 2001.

[35] T. E. Colanzi, W. K. Assuncão, S. R. Vergilio, P. R. Farah, and G. Guizzo, "The symposium on search-based software engineering: Past, present and future," *Inf. Softw. Technol.*, vol. 127, 2020, Art. no. 106372.

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.

[36] M. Harman, S. A. Mansouri, and Y. Zhang, "Search-based software engineering: Trends, techniques and applications," *ACM Comput. Surv.*, vol. 45, no. 1, pp. 11:1–11:61, Dec. 2012.

[37] W. K. Assuncao et al., "Analysis of a many-objective optimization approach for identifying microservices from legacy systems," *Empirical Softw. Eng. (EMSE)*, vol. 27, 2022, Art. no. 51.

[38] R. M. de Mello, "Conceptual framework for supporting the identification of representative samples for surveys in software engineering," Ph.D. dissertation, Federal Univ. Rio de Janeiro, Rio de Janeiro, Brazil, 2016.

[39] U. Van Heesch, T. Theunissen, O. Zimmermann, and U. Zdun, "Software specification and documentation in continuous software development: A focus group report," in *Proc. 22nd Eur. Conf. Pattern Lang. Programs*, 2017, pp. 1–13.

[40] L. F. Junionello, R. de Mello, R. Oliveira, L. Sousa, A. López, and A. Garcia, "Revealing developers' arguments on validating the incidence of code smells: A focus group experience," in *Proc. Anais do IX Workshop de Visualizacão, Evolucão e Manutencão de Softw.*, Porto Alegre, Brazil: SBC, 2021, pp. 31–35.

[41] R. de Mello et al., "Recommendations for developers identifying code smells," *IEEE Softw.*, vol. 40, no. 2, pp. 90–98, Mar./Apr. 2023.

[42] A. Arcuri and L. Briand, "A Hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Softw. Testing, Verification Rel.*, vol. 24, no. 3, pp. 219–250, 2014.

[43] T. E. Colanzi, W. K. G. Assuncão, P. R. Farah, S. R. Vergilio, and G. Guizzo, "A review of ten years of the symposium on search-based software engineering," in *Search-Based Software* Engineering. New York, NY, USA: Springer-Verlag, 2019, pp. 42–57.

[44] W. K. Assuncão, J. Krüger, and W. D. Mendonca, "Variability management meets microservices: Six challenges of re-engineering microservice-based webshops," in *Proc. 24th ACM Conf. Syst. Softw. Product Line: Vol. A*, 2020, pp. 1–6.

[45] R. Nakazawa, T. Ueda, M. Enoki, and H. Horii, "Visualization tool for designing microservices with the monolith-first approach," in *Proc. IEEE Work. Conf. Softw. Vis.*, 2018, pp. 32–42.

[46] C. Wohlin, P. Runeson, M. Host, M. Ohlsson, B. Regnell, and A. Wesslen, *Experimentation in Software Engineering: An Introduction*, Berlin, Germany: Springer, 2000.

[47] H. Knoche and W. Hasselbring, "Using microservices for legacy software modernization," *IEEE Softw.*, vol. 35, no. 3, pp. 44–49, May/Jun. 2018.

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 21,2026 at 22:36:09 UTC from IEEE Xplore. Restrictions apply.